

基于生成式检索的推荐系统

Shashank Rajput*

University of Wisconsin-Madison

Nikhil Mehta*

Google DeepMind

Anima Singh

Google DeepMind

Raghunandan Keshavan

Google

Trung Vu

Google

Lukasz Heldt

Google

Lichan Hong

Google DeepMind

Yi Tay

Google DeepMind

Vinh Q. Tran

Google

Jonah Samost

Google

Maciej Kula

Google DeepMind

Ed H. Chi

Google DeepMind

Maheswaran Sathiamoorthy

Google DeepMind

Abstract

现代推荐系统通过将查询和候选物品嵌入同一个统一空间来执行大规模检索，随后进行近似最近邻搜索，根据查询嵌入选出排名靠前的候选项。本文提出一种新颖的生成式检索方法，其中检索模型以自回归方式解码目标候选选项的标识符。为此，我们为每个物品创建具有语义含义的码字元组，作为其语义 ID。给定用户会话中物品的语义 ID，我们训练一个基于 Transformer 的序列到序列模型，以预测用户接下来将与之交互的物品的语义 ID。我们表明，采用所提出范式训练的推荐系统在多个数据集上显著优于当前 SOTA 模型。此外，我们还表明，将语义 ID 融入序列到序列模型可增强其泛化能力，这一点体现在模型对没有历史交互记录的物品取得了更好的检索性能。

1 引言

推荐系统帮助用户发现感兴趣的内容，并广泛应用于视频 [4, 43, 9]、应用 [3]、商品 [6, 8] 和音乐 [18, 19] 等各种推荐领域。现代推荐系统采用检索后排序的策略，即在检索阶段选出

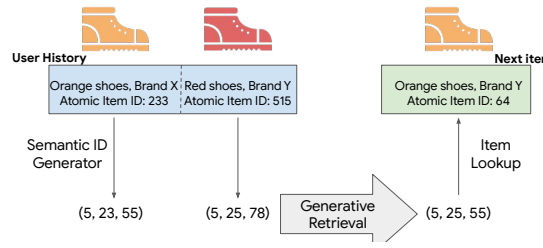


图 1: 生成式推荐器的 Transformer 索引 (TIGER) 框架概览。在 TIGER 中，每个物品均表示为一个离散语义词元元组，从而将序列推荐表述为生成式检索任务。

一组可行的候选项，随后使用排序模型对其进行排序。由于排序模型只能处理其接收到的候选项，因此我们希望检索阶段能够输出高度相关的候选项。

构建检索模型已有标准且成熟的方法。

矩阵分解 [19] 在同一空间中学习查询和候选项的嵌入。为了更好地捕捉数据中的非线性特征，近年来，采用内积将查询和候选项嵌入到同一空间的双编码器架构 [39] (即一个塔用于查询，另一个塔用于候选项) 日益流行。在推理时使用这些模型，需要通过候选项塔创建一个存储所有物品嵌入的索引。对于给定查询，先使用查询塔获得其嵌入，再使用近似最近邻 (ANN) 算法进行检索。近年来，双编码器架构还被扩展用于序列推荐 [11, 24, 41, 17, 32, 6, 44]，显式考虑用户与物品交互的顺序。

我们提出了一种为序列推荐构建生成式检索模型的新范式。不同于传统的查询—候选项匹配方法，我们的方法使用端到端生成模型直接预测候选项 ID。我们提出利用 Transformer [36] 的记忆 (参数) 作为推荐系统检索的端到端索引，这与 Tay 等人 [34] 使用 Transformer 记忆进行文档检索的做法相呼应。我们将该方法称为 *Transformer Index for GEnenerative Recommenders* (TIGER)。图 1 展示了 TIGER 的高层概览。TIGER 的独特之处在于一种称为“语义 ID”的新型物品语义表示，它是由每个物品的内容信息派生出的一系列词元。具体而言，给定物品的文本特征，我们使用预训练文本编码器 (例如 SentenceT5 [27]) 生成稠密内容嵌入。随后对物品嵌入应用量化方案，形成一组有序词元/码字，我们将其称为物品的语义 ID。最终，这些语义 ID 被用于在序列推荐任务上训练 Transformer 模型。

将物品表示为语义词元序列具有诸多优势。在具有语义含义的数据上训练 Transformer 记忆，可使相似物品之间共享知识。这样，我们便无需使用此前在推荐模型中作为物品特征的原子化随机物品 ID [33, 42, 11, 8]。使用物品的语义词元表示后，模型不易受到推荐系统固有反馈循环 [1, 26, 39] 的影响，因而能够泛化到语料库中新加入的物品。此外，使用词元序列表示物品有助于缓解物品语料库规模带来的挑战；词元所能表示的物品数量等于序列中各词元基数的乘积。通常，物品语料库规模可达数十亿量级，为每个物品学习唯一嵌入会消耗大量内存。尽管可以采用基于随机哈希的技术 [16] 来缩小物品表示空间，但在本工作中，我们表明使用具有语义含义的词元表示物品是一种颇具吸引力的替代方案。本工作的主要贡献总结如下：

1. 我们提出 TIGER，这是一种新颖的生成式检索推荐框架，它为每个物品分配语义 ID，并训练检索模型来预测给定用户可能与之交互的物品的语义 ID。
2. 我们表明，以召回率和 NDCG 指标衡量时，TIGER 在多个数据集上优于现有 SOTA 推荐系统。

3. 我们发现, 这种新的生成式检索范式为序列推荐系统带来了两项额外能力: 1. 推荐新物品和低频物品的能力, 从而改善冷启动推荐; 2. 使用可调参数生成多样化推荐的能力。

论文概览。在 Section 2 中, 我们简要回顾推荐系统、生成式检索以及本文所采用的语义 ID 生成技术。在 Section 3 中, 我们介绍所提出的框架, 并概述用于生成语义 ID 的各种技术。我们在 Section 4 中给出实验结果, 并在 Section 5 中总结全文。

2 相关工作

序列推荐器。在推荐系统中使用深度序列模型已经形成了丰富的研究成果。GRU4REC [11] 是首个将基于 GRU 的 RNN 用于序列推荐的模型。Li 等人 [24] 提出了神经注意力会话推荐 (NARM), 其中使用注意力机制和 GRU 层来追踪用户的长期意图。Zhang 等人提出的 AttRec [41] 使用自注意力机制对用户当前会话中的意图进行建模, 并通过度量学习建模用户-物品亲和度来实现个性化。与此同时, Kang 等人也提出了 SASRec [17], 其采用了类似于仅解码器 Transformer 模型的自注意力机制。受掩码语言建模在语言任务中取得成功的启发, BERT4Rec [32] 和 Transformers4Rec [6] 将带有掩码策略的 Transformer 模型用于序列推荐任务。S³-Rec [44] 不仅采用掩码, 还在四个自监督任务上进行预训练, 以改进数据表示。上述模型为每个物品学习一个高维嵌入, 并在最大内积搜索 (MIPS) 空间中执行 ANN, 以预测下一个物品。相比之下, 我们提出的 TIGER 技术使用生成式检索直接预测下一个物品的语义 ID。

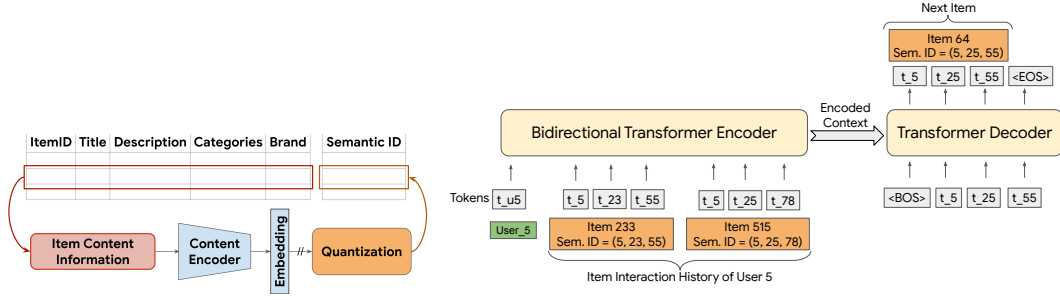
P5 [8] 针对多任务推荐系统微调预训练大语言模型。P5 模型依赖 LLM 分词器 (SentencePiece 分词器 [29]), 从随机分配的物品 ID 中生成词元。而我们使用基于物品内容信息学习得到的物品语义 ID 表示。在实验 (Table 2) 中, 我们证明, 基于物品语义 ID 表示的推荐系统比使用随机编码取得了好得多的结果。

语义 ID。Hou 等人提出 VQ-Rec [12], 利用内容信息生成用于物品表示的“编码”(类似于语义 ID)。然而, 他们关注的是构建可迁移的推荐系统, 并未以生成方式使用这些编码进行检索。尽管他们也使用乘积量化 [15] 生成编码, 但我们使用 RQ-VAE 生成语义 ID, 从而得到物品的层次化表示 (第 4.2 节)。在与我们同期开展的一项工作中, Singh 等人 [31] 表明, 在大规模推荐系统中用层次化语义 ID 替代排序模型的物品 ID, 可以提升模型的泛化能力。

生成式检索。尽管过去已经提出了学习搜索索引的技术 [20], 生成式检索却是一种近期发展起来的文档检索方法, 其任务是从数据库中返回一组相关文档。相关示例包括 GENRE [5]、DSI [34]、NCI [37] 和 CGR [22]。附录 A 对相关工作进行了更详细的介绍。据我们所知, 我们是首个提出使用物品语义 ID 表示进行推荐系统生成式检索的工作。

3 所提出的框架

我们提出的框架包含两个阶段:



(a) 使用内容嵌入量化为物品生成语义 ID (b) 用于构建生成式检索序列到序列模型的基于 Transformer 的编码器—解码器设置。

图 2: TIGER 所采用建模方法的概览。

1. 利用内容特征生成语义 ID。该阶段将物品内容特征编码为嵌入向量，并将嵌入量化为语义码字元组。所得码字元组称为物品的语义 ID。
2. 在语义 ID 上训练生成式推荐系统。使用语义 ID 序列，在序列推荐任务上训练 Transformer 模型。

3.1 语义 ID 生成

本节介绍推荐语料库中物品的语义 ID 生成过程。我们假设每个物品都有能够捕获有用语义信息的相关内容特征（例如标题、描述或图像）。此外，我们假设可以使用预训练内容编码器生成语义嵌入 $\mathbf{x} \in \mathbb{R}^d$ 。例如，可使用 Sentence-T5 [27] 和 BERT [7] 等通用预训练文本编码器转换物品的文本特征，从而获得语义嵌入。随后对语义嵌入进行量化，为每个物品生成语义 ID。图 2a 给出了该过程的高层概览。

我们将语义 ID 定义为长度为 m 的码字元组。元组中的每个码字来自不同的码本。因此，语义 ID 能够唯一表示的物品数量等于各码本大小的乘积。尽管不同的语义 ID 生成技术会使 ID 具有不同的语义性质，但我们希望它们至少具备以下性质：相似物品（内容特征相似或语义嵌入接近的物品）应具有部分重叠的语义 ID。例如，语义 ID 为 (10, 21, 35) 的物品与语义 ID 为 (10, 21, 40) 的物品之间，应当比其与 ID 为 (10, 23, 32) 的物品之间更为相似。接下来，我们讨论用于生成语义 ID 的量化方案。

用于语义 ID 的 RQ-VAE。残差量化变分自编码器（Residual-Quantized Variational AutoEncoder, RQ-VAE）[40] 是一种多层向量量化器，它对残差进行量化，从而生成一个码字元组（即语义 ID）。该自编码器通过更新量化码本及 DNN 编码器—解码器参数进行联合训练。Fig. 3 展示了通过残差量化生成语义 ID 的过程。

RQ-VAE 首先通过编码器 $\mathcal{E}$ 对输入 $\mathbf{x}$ 进行编码，以学习潜在表示 $\mathbf{z} := \mathcal{E}(\mathbf{x})$ 。在第零层 ($d = 0$)，初始残差直接定义为 $\mathbf{r}_0 := \mathbf{z}$ 。在每一层 d ，都有一个码本 $\mathcal{C}_d := \{\mathbf{e}_k\}_{k=1}^K$ ，其中 K 为码本大小。随后，通过将 $\mathbf{r}_0$ 映射到该层码本中最近的嵌入来对其进行量化。在 $d = 0$ 处与其最接近的嵌入 $\mathbf{e}_{c_0}$ 的索引，即 $c_0 = \arg \min_i \|\mathbf{r}_0 - \mathbf{e}_i\|$ ，表示第零个码字。对于下一层 $d = 1$ ，残差定义为 $\mathbf{r}_1 := \mathbf{r}_0 - \mathbf{e}_{c_0}$ 。然后，与第零层类似，通过在第

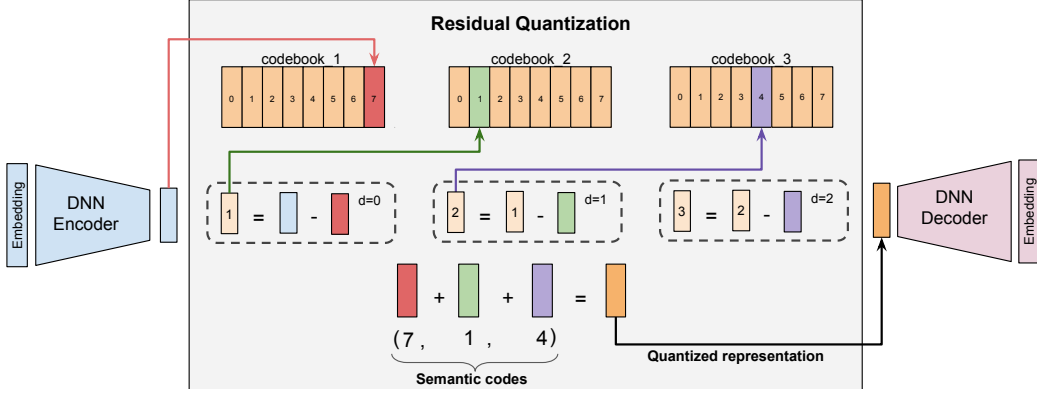


图 3: RQ-VAE: 图中, DNN 编码器输出的向量 (记为 r_0 , 以蓝色条表示) 被送入以迭代方式工作的量化器。首先, 在第一层码本中找到与 r_0 最接近的向量。设该最近向量为 e_{c_0} (以红色条表示)。随后, 将残差计算为 $r_1 := r_0 - e_{c_0}$ 。该残差被送入量化器的第二层, 并重复上述过程: 在第二层中找到与 r_1 最接近的向量, 记为 e_{c_1} (以绿色条表示), 然后将第二层残差计算为 $r_2 = r_1 - e_{c_1}$ 。接着, 在 r_2 上第三次重复这一过程。语义编码由 e_{c_0}, e_{c_1} , 和 e_{c_2} 在各自码本中的索引得到。在图示示例中, 最终得到编码 (7, 1, 4)。

一层码本中寻找与 r_1 最近的嵌入来计算第一层的编码。递归重复该过程 m 次, 便得到由 m 个码字组成、表示语义 ID 的元组。这种递归方法以由粗到细的粒度逼近输入。需要注意的是, 我们选择为 m 层中的每一层分别使用大小为 K 的独立码本, 而不是使用单个大小为 mK 的码本。这是因为残差的范数往往随层数增加而减小, 因此不同层可以采用不同的粒度。

得到语义 ID ($c_0, \dots, c_{m-1}$) 后, 将 z 的量化表示计算为 $\hat{z} := \sum_{d=0}^{m-1} e_{c_d}$ 。随后, 将 $\hat{z}$ 送入解码器, 解码器尝试利用 $\hat{z}$ 重建输入 x 。RQ-VAE 的损失定义为 $\mathcal{L}(x) := \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{rqvae}}$, 其中 $\mathcal{L}_{\text{recon}} := \|x - \hat{x}\|^2$, 且 $\mathcal{L}_{\text{rqvae}} := \sum_{d=0}^{m-1} \|\text{sg}[r_d] - e_{c_d}\|^2 + \beta \|r_d - \text{sg}[e_{c_d}]\|^2$ 。这里, $\hat{x}$ 是解码器的输出, sg 是停止梯度操作 [35]。该损失对编码器、解码器和码本进行联合训练。

按照 [40] 中提出的方法, 为防止 RQ-VAE 出现码本坍塌, 即大多数输入仅被映射到少量码本向量, 我们使用基于 k-means 聚类的码本初始化。具体而言, 我们在第一个训练批次上应用 k-means 算法, 并使用聚类中心进行初始化。

其他量化方案。生成语义 ID 的一种简单替代方案是使用局部敏感哈希 (LSH)。我们在第 4.2 小节中进行了消融研究, 发现 RQ-VAE 确实优于 LSH。另一种选择是以层次化方式使用 k-means 聚类 [34], 但这种方法会丢失不同聚类之间的语义关系 [37]。我们也尝试了 VQ-VAE; 虽然它在检索时生成候选项方面与 RQ-VAE 表现相近, 但会失去 ID 的层次结构, 而这种层次结构赋予了第 4.3 节所讨论的许多新能力。

处理冲突。根据语义嵌入的分布、码本大小的选择以及码字长度, 可能会发生语义冲突 (*i.e.*, 即多个物品可能映射到同一个语义 ID)。为消除冲突, 我们在有序语义编码末尾附加一个额外词元, 使其保持唯一。例如, 如果两个物品共享语义 ID (12, 24, 52), 我们会附加额外词元加以区分, 将两个物品分别表示为 (12, 24, 52, 0) 和 (12, 24, 52, 1)。为

检测冲突，我们维护一张将语义 ID 映射到相应物品的查找表。需要注意的是，冲突检测和修复仅在 RQ-VAE 模型训练完成后执行一次。此外，由于语义 ID 是整数元组，与高维嵌入相比，该查找表在存储方面更为高效。

3.2 使用语义 ID 的生成式检索

我们按交互时间顺序对每位用户交互过的物品进行排序，从而构建物品序列。随后，给定形如 $(\text{item}_1, \dots, \text{item}_n)$ 的序列，推荐系统的任务是预测下一个物品 item_{n+1} 。我们提出一种生成式方法，直接预测下一个物品的语义 ID。形式化地，令 $(c_{i,0}, \dots, c_{i,m-1})$ 表示 item_i 的长度为 m 的语义 ID。然后，我们将物品序列转换为序列 $(c_{1,0}, \dots, c_{1,m-1}, c_{2,0}, \dots, c_{2,m-1}, \dots, c_{n,0}, \dots, c_{n,m-1})$ 。接着训练序列到序列模型，以预测 item_{n+1} 的语义 ID，即 $(c_{n+1,0}, \dots, c_{n+1,m-1})$ 。鉴于本框架的生成性质，解码器生成的语义 ID 可能无法与推荐语料库中的任何物品匹配。然而，正如附录 (Fig. 6) 所示，此类事件发生的概率很低。我们在附录 E 中进一步讨论了如何处理此类事件。

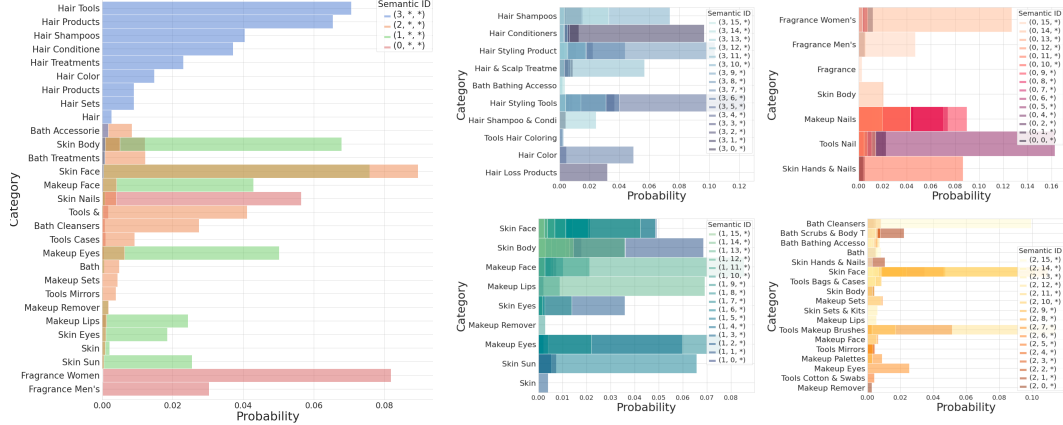
4 实验

数据集。我们在 Amazon Product Reviews 数据集 [10] 中的三个公开真实世界基准上评估所提出的框架，该数据集包含 1996 年 5 月至 2014 年 7 月的用户评论和物品元数据。具体而言，我们使用 Amazon Product Reviews 数据集的三个类别进行序列推荐任务：“Beauty”、“Sports and Outdoors”和“Toys and Games”。数据集统计信息和预处理见附录 C。

评估指标。我们使用 top-k 召回率 (Recall@K) 和归一化折损累计增益 (NDCG@K)，其中 $K = 5, 10$ ，来评估推荐性能。

RQ-VAE 实现细节。如第 3.1 节所述，RQ-VAE 用于量化物品的语义嵌入。我们使用预训练 Sentence-T5 [27] 模型获取数据集中每个物品的语义嵌入。具体而言，我们使用物品的标题、价格、品牌和类别等内容特征构造句子，然后将其输入预训练 Sentence-T5 模型，得到物品的 768 维语义嵌入。

RQ-VAE 模型由三个组件构成：将输入语义嵌入编码为潜在表示的 DNN 编码器、输出量化表示的残差量化器，以及将量化表示解码回语义输入嵌入空间的 DNN 解码器。编码器包含三个大小分别为 512、256 和 128、采用 ReLU 激活的中间层，最终潜在表示的维度为 32。为量化该表示，我们进行三级残差量化。每一级维护一个基数为 256 的码本，其中码本内每个向量的维度为 32。计算总损失时，我们使用 $\beta = 0.25$ 。RQ-VAE 模型训练 20k 个 epoch，以确保较高的码本使用率 ($\geq 80\%$)。我们使用 Adagrad 优化器，学习率为 0.4，批大小为 1024。训练完成后，我们使用学得的编码器和量化组件为每个物品生成三元组语义 ID。为避免多个物品映射到同一语义 ID，我们为前三个码字相同的物品添加唯一的 4^{th} 编码，*i.e.* 与元组 (7, 1, 4) 关联的两个物品分别被分配为 (7, 1, 4, 0) 和 (7, 1, 4, 1) (若不存在冲突，我们仍将第四个码字指定为 0)。这样，推荐



(a) 数据集中所有物品的真实类别分布, (b) 语义 ID 为 $(c_1, *, *)$ 的物品类别分布, 其中 $c_1 \in \{1, 2, 3, 4\}$ 。类别按照第二个语义 token c_2 的取值着色。

图 4: Amazon Beauty 数据集上 RQ-VAE 语义 ID (c_1, c_2, c_3, c_4) 的定性研究。真实类别分布于不同语义 token 之中; 同时, RQ-VAE 语义 ID 形成物品层次结构, 其中第一个语义 token (c_1) 对应粗粒度类别, 第二、第三个语义 token (c_2/c_3) 对应细粒度类别。

语料库中的每个物品都获得一个长度为 4 的唯一语义 ID。

序列到序列模型实现细节。我们使用开源 T5X 框架 [28] 实现基于 Transformer 的编码器—解码器架构。为了使模型能够处理序列推荐任务的输入, 序列到序列模型的词表包含每个语义码字对应的词元。具体而言, 词表包含 1024 (256×4) 个用于表示语料库中物品的词元。除物品的语义码字外, 我们还向词表中加入用户特定词元。为限制词表大小, 我们仅为用户 ID 添加 2000 个词元。我们使用哈希技巧 [38] 将原始用户 ID 映射到 2000 个用户 ID 词元之一。我们将输入序列构造为用户 ID 词元, 后接与给定用户物品交互历史相对应的语义 ID 词元序列。我们发现, 在输入中加入用户 ID 能使模型对检索到的物品进行个性化。

基于 Transformer 的编码器和解码器模型均使用 4 层, 每层包含 6 个维度为 64 的自注意力头。所有层均使用 ReLU 激活函数。MLP 维度和输入维度分别设为 1024 和 128。我们使用 0.1 的 dropout。模型总计约有 1300 万个参数。对于“Beauty”和“Sports and Outdoors”数据集, 我们将该模型训练 200k 步。由于“Toys and Games”数据集规模较小, 仅训练 100k 步。批大小为 256。前 10k 步的学习率为 0.01, 此后采用平方根倒数衰减调度。

4.1 序列推荐性能

在本节中, 我们将所提出的生成式检索框架与以下序列推荐方法进行比较 (这些方法在附录 B 中有简要介绍): GRU4Rec [11]、Caser [33]、HGN [25]、SASRec [17]、BERT4Rec [32]、FDSA [42]、S³-Rec [44] 和 P5 [8]。值得注意的是, 除 P5 外, 所有基线方法都使用双编码器学习高维向量空间, 将用户过去的物品交互和候选物品编码为高维表示, 并利用最大内积搜索 (MIPS) 检索用户下一步可能交互的候选物品。相

表 1: 序列推荐性能比较。最后一行给出了 TIGER 相对于最佳基线的提升百分比。粗体（下划线）表示最佳（次佳）指标。

方法	运动与户外				美妆				玩具与游戏			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
P5 [8]	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
Caser [33]	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN [25]	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec [11]	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec [32]	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA [42]	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec [17]	0.0233	0.0154	0.0350	0.0192	0.0387	<u>0.0249</u>	0.0605	0.0318	<u>0.0463</u>	<u>0.0306</u>	0.0675	0.0374
S ³ -Rec [44]	<u>0.0251</u>	<u>0.0161</u>	<u>0.0385</u>	<u>0.0204</u>	<u>0.0387</u>	0.0244	<u>0.0647</u>	<u>0.0327</u>	0.0443	0.0294	<u>0.0700</u>	<u>0.0376</u>
TIGER [本文]	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432
	+5.22%	+12.55%	+3.90%	+10.29%	+17.31%	+29.04%	+0.15%	+17.43%	+12.53%	+21.24%	+1.71%	+14.97%

比之下，我们的新型生成式检索框架使用序列到序列模型，逐词元直接预测物品的语义 ID。

推荐性能。我们在序列推荐任务上对所提出的 TIGER 进行了广泛分析，并与上述基线进行比较。除 P5 外，所有基线的结果均取自 Zhou 等人 [44] 公开提供的结果³。对于 P5，我们使用作者提供的源代码。然而，为了公平比较，我们更新了数据预处理方法，使其与其他基线和我们方法保持一致。附录 D 提供了有关这些改动的更多细节。

结果如表 1 所示。我们观察到，TIGER 始终优于现有基线⁴。在所考虑的三个基准上，我们都看到了显著提升。尤其是在 Beauty 基准上，相较于次佳基线，TIGER 的表现明显更好：与 SASRec 相比，NDCG@5 最多提升 29%；与 S³-Rec 相比，Recall@5 提升 17.3%。类似地，在 Toys and Games 数据集上，TIGER 的 NDCG@5 和 NDCG@10 分别提升了 21% 和 15%。

4.2 物品表示

本节分析 RQ-VAE 语义 ID 的若干重要特性。首先，我们进行定性分析，以观察语义 ID 的层次性质。接下来，通过与另一种基于哈希的量化方法对比性能，我们评估采用 RQ-VAE 进行量化这一设计选择的重要性。最后，我们开展消融研究，通过将 TIGER 与使用随机 ID 表示物品的序列到序列模型进行比较，研究使用语义 ID 的重要性。

定性分析。我们在图 4 中分析了为 Amazon Beauty 数据集学习到的 RQ-VAE 语义 ID。为便于说明，我们将 RQ-VAE 的层数设为 3，各层码本大小分别为 4、16 和 256，即对于某个物品给定的语义 ID (c_1, c_2, c_3) ， $0 \leq c_1 \leq 3$ 、 $0 \leq c_2 \leq 15$ 和 $0 \leq c_3 \leq 255$ 。在图 4a 中，我们使用 c_1 标注每个物品的类别，以在数据集整体类别分布中可视化 c_1 特定的类别。如图 4a 所示， c_1 捕获物品的高层类别。例如， $c_1 = 3$ 包含了大多数与“Hair”相关的产品。类似地，具有 $c_1 = 1$ 的物品大多是面部、唇部和眼部的“Makeup”与“Skin”产品。

³<https://github.com/aHuiWang/CIKM2020-S3Rec>

⁴我们在表 9 中表明，TIGER 各项指标的标准误差可以忽略不计。

表 2: 生成式检索中不同 ID 生成技术的消融研究。结果表明, RQ-VAE 语义 ID (SID) 的性能显著优于哈希 SID 和随机 ID。

方法	运动与户外				美妆				玩具与游戏			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
随机 ID	0.007	0.005	0.0116	0.0063	0.0296	0.0205	0.0434	0.0250	0.0362	0.0270	0.0448	0.0298
LSH SID	0.0215	0.0146	0.0321	0.0180	0.0379	0.0259	0.0533	0.0309	0.0412	0.0299	0.0566	0.0349
RQ-VAE SID	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432

我们还通过固定 c_1 , 并在 Fig. 4b 中可视化 c_2 所有可能取值下的类别分布, 展示 RQ-VAE 语义 ID 的层次性质。我们再次发现, 第二个码字 c_2 会将 c_1 捕获的高层语义进一步划分为细粒度类别。RQ-VAE 学习到的语义 ID 所具有的层次性质带来了广泛的新能力, 我们将在第 4.3 节中讨论。不同于基于随机原子 ID 学习物品嵌入的现有推荐系统, TIGER 使用语义 ID, 使语义相似的物品具有重叠码字, 从而让模型能够有效共享数据集中语义相似物品的知识。

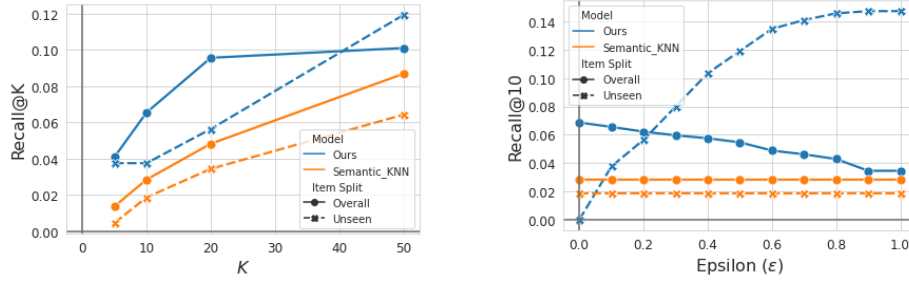
哈希与 RQ-VAE 语义 ID。我们通过比较 RQ-VAE 与用于生成语义 ID 的局部敏感哈希 (LSH) [14, 13, 2], 研究 RQ-VAE 在框架中的重要性。LSH 是一种常用哈希技术, 很容易适配我们的设定。为了生成 LSH 语义 ID, 我们使用 h 个随机超平面 $w_1, \dots, w_h$ 对嵌入向量 x 进行随机投影, 并计算如下二进制向量: $(1_{w_1^\top x > 0}, \dots, 1_{w_h^\top x > 0})$ 。该向量被转换为整数编码 $c_0 = \sum_{i=1}^h 2^{i-1} 1_{w_i^\top x > 0}$ 。使用相互独立的随机超平面集合将这一过程重复 m 次, 得到 m 个码字 $(c_0, c_1, \dots, c_{m-1})$, 我们将其称为 LSH 语义 ID。

在表 2 中, 我们比较了 LSH 语义 ID 与所提出的 RQ-VAE 语义 ID 的性能。在该实验中, 对于 LSH 语义 ID, 我们使用 $h = 8$ 个随机超平面, 并设置 $m = 4$, 以确保其基数与 RQ-VAE 相当。超平面的参数从标准正态分布中随机采样, 从而保证超平面具有球对称性。结果表明, RQ-VAE 始终优于 LSH。这说明, 在给定相同的基于内容的语义嵌入时, 与使用随机投影相比, 通过非线性深度神经网络 (DNN) 架构学习语义 ID 能够获得更好的量化效果。

随机 ID 与语义 ID。我们还比较了语义 ID 在生成式检索推荐系统中的重要性。具体而言, 我们将随机生成的 ID 与语义 ID 进行比较。为生成随机 ID 基线, 我们为每个物品分配 m 个随机码字。某个物品长度为 m 的随机 ID 就是 $(c_1, \dots, c_m)$, 其中 c_i 从 $\{1, 2, \dots, K\}$ 中均匀随机采样。我们为随机 ID 基线设置 $m = 4$ 和 $K = 255$, 使其基数与 RQ-VAE 语义 ID 相近。表 2 展示了随机 ID 与 RQ-VAE、LSH 语义 ID 的比较。可以看到, 语义 ID 始终优于随机 ID 基线, 这凸显了利用基于内容的语义信息的重要性。

4.3 新能力

我们介绍由所提出的生成式检索框架直接带来的两项新能力, 即冷启动推荐和推荐多样性。我们将这些能力称为“新”能力, 是因为现有序列推荐模型 (见第 4.1 节中的基线) 无法直接用于满足这些真实世界用例。这些能力源于基于 RQ-VAE 的语义 ID 与本框架生成式检索方法之间的协同作用。以下各节将讨论 TIGER 如何用于这些场景。



(a) Recall@K 与 K 的关系 ($\epsilon = 0.1$)。

(b) Recall@10 与 ϵ 的关系。

图 5: 冷启动检索场景下的性能。

冷启动推荐。本节研究所提出框架的冷启动推荐能力。由于真实世界推荐语料库变化迅速，新物品会不断加入。新加入的物品在训练语料库中缺少用户曝光，因此，使用随机原子 ID 表示物品的现有推荐模型无法将新物品检索为潜在候选项。相比之下，TIGER 框架在预测下一个物品时利用了物品语义，因此能够轻松进行冷启动推荐。

在该分析中，我们使用 Amazon Reviews 的 Beauty 数据集。为模拟新加入的物品，我们从训练数据划分中移除 5% 的测试物品，并将这些被移除的物品称为未见物品。从训练划分中移除这些物品可确保不存在与未见物品有关的数据泄漏。与之前一样，我们使用长度为 4 的语义 ID 表示物品，其中前 3 个词元由 RQ-VAE 生成，4th 词元用于确保所有已见物品都具有唯一 ID。我们在训练划分上训练 RQ-VAE 量化器和序列到序列模型。训练完成后，我们使用 RQ-VAE 模型为数据集中的所有物品生成语义 ID，包括物品语料库中的所有未见物品。

给定模型预测的语义 ID (c_1, c_2, c_3, c_4)，我们检索具有相同对应 ID 的已见物品。请注意，根据定义，模型预测的每个语义 ID 最多只能与训练数据集中的物品匹配。此外，将前三个语义词元相同的未见物品 *i.e.* (c_1, c_2, c_3) 加入检索候选项列表。最后，在检索一组 top-K 候选项时，我们引入超参数 ϵ ，用于指定框架所选未见物品的最大比例。

我们在 Fig. 5 中比较了 TIGER 与 k 近邻 (KNN) 方法在冷启动推荐场景下的性能。对于 KNN，我们在语义表示空间中执行最近邻搜索，并将该基于 KNN 的基线称为 Semantic_KNN。Fig. 5a 表明，采用 $\epsilon = 0.1$ 的框架在所有 Recall@K 指标上始终优于 Semantic_KNN。在 Fig. 5b 中，我们比较了本方法与 Semantic_KNN 在不同 ϵ 取值下的表现。在 $\epsilon \geq 0.1$ 的所有设置下，本方法均优于基线。

推荐多样性。尽管 Recall 和 NDCG 是评估推荐系统的主要指标，预测多样性也是另一个至关重要的目标。多样性较差的推荐系统可能不利于用户的长期参与。在此，我们讨论如何利用生成式检索框架预测多样化的物品。我们表明，解码过程中的基于温度的采样能够有效控

表 3: 模型在 Beauty 数据集上预测的类别分布熵。熵越高，表示模型预测的物品越多样。

温度	Entropy@10	Entropy@20	Entropy@50
T = 1.0	0.76	1.14	1.70
T = 1.5	1.14	1.52	2.06
T = 2.0	1.38	1.76	2.28

制模型预测的多样性。虽然基于温度的采样可以应用于任何现有推荐模型，但由于 RQ-VAE 语义 ID 的特性，TIGER 能够在不同层次上进行采样。例如，对语义 ID 的第一个词元进行采样，可以检索粗粒度类别中的物品；而对第二个或第三个词元进行采样，则可以在某个类别内部采样物品。

表 4: 使用基于温度的解码时的推荐多样性。

目标类别	预测排名前 10 的物品中最常见的类别	
	T = 1.0	T = 2.0
美发造型产品	美发造型产品	美发造型产品、美发造型工具、面部护肤
美甲工具	美甲工具	美甲工具、指甲彩妆
指甲彩妆	指甲彩妆	指甲彩妆、手部与指甲护肤、美甲工具
眼部护肤	眼部护肤	头发拉直产品、面部护肤、美发造型产品、眼部护肤
面部彩妆	彩妆刷具、面部彩妆	彩妆刷具、面部彩妆、面部护肤、彩妆套装、美发造型工具
防脱发产品	防脱发产品、面部护肤、身体护肤	面部护肤、防脱发产品、洗发产品、头发与头皮护理、护发素

我们使用 Entropy@K 指标定量衡量预测的多样性，其中熵是根据模型所预测 top-K 物品真实类别的分布计算得到的。表 3 报告了不同温度值下的 Entropy@K。我们观察到，在解码阶段进行温度采样能够有效提升物品真实类别的多样性。我们还在表 4 中进行了定性分析。

4.4 消融研究

我们在表 5 中衡量了改变序列到序列模型层数的影响。可以看到，随着网络规模增大，各项指标略有提升。我们还衡量了提供用户信息的影响，相关结果见附录中的表 8。

4.5 无效 ID

由于模型以自回归方式解码目标语义 ID 的码字，因此模型可能预测出无效 ID（即无法映射到推荐数据集中任何物品的 ID）。在实验中，我们使用长度为 4 的语义 ID，每个码字的基数为 256（即每层的码本大小为 256）。这种组合覆盖的可能 ID 数量为 $= 256^4$ ，约为 4 万亿。另一方面，我们所考虑数据集中的物品数量为 10K–20K（见表 6）。尽管有效 ID 仅占完整 ID 空间的一小部分，但我们观察到模型几乎总能预测出有效 ID。图 6 展示了 TIGER 生成的无效 ID 比例随检索物品数量 K 的变化。对于 top-10 预测，三个数据集上的无效 ID 比例为 $\sim 0.1\% - 1.6\%$ 。为抵消无效 ID 的影响并始终得到 top-10 个有效 ID，我们可以增大束搜索宽度并过滤无效 ID。

表 5: 不同层数下的 Recall 和 NDCG 指标。

层数	Recall@5	NDCG@5	Recall@10	NDCG@10
3	0.04499	0.03062	0.06699	0.03768
4	0.0454	0.0321	0.0648	0.0384
5	0.04633	0.03206	0.06596	0.03834

需要强调的是，尽管会生成无效 ID，TIGER 与其他常用序列推荐方法相比仍实现了最先进的性能。一种处理无效词元的扩展方案是：当模型生成无效词元时进行前缀匹配。对语义 ID 进行前缀匹配，可以检索与模型生成词元具有相似语义的物品。鉴于 RQ-VAE 词元具有层次结构，前缀匹配可被视为模型预测物品类别，而非物品索引。需要注意的是，这种扩展还可能进一步提升 Recall/NDCG 指标。我们将其留作未来工作。

5 结论

本文提出了一种名为 TIGER 的新范式，使用生成模型在推荐系统中检索候选物品。该方法的基础是一种新颖的物品语义 ID 表示，它在内容嵌入上使用层次量化器（RQ-VAE）生成构成语义 ID 的词元。我们的框架得到的模型无需创建索引即可训练和提供服务——Transformer 的记忆充当物品的语义索引。值得注意的是，我们的嵌入表基数不会随物品空间基数线性增长；与训练期间需要创建大型嵌入表，或为每个物品单独生成索引的系统相比，这一点对我们有利。通过在三个数据集上的实验，我们表明，该模型能够达到 SOTA 检索性能，同时泛化到新的、未见过的物品。

References

- [1] Himan Abdollahpour, Masoud Mansoury, Robin Burke, and Bamshad Mobasher. The unfairness of popularity bias in recommendation. *arXiv preprint arXiv:1907.13286*, 2019.
- [2] Moses S Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing*, pages 380–388, 2002.
- [3] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*, pages 7–10, 2016.
- [4] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*, pages 191–198, 2016.
- [5] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. *arXiv preprint arXiv:2010.00904*, 2020.
- [6] Gabriel de Souza Pereira Moreira, Sara Rabhi, Jeong Min Lee, Ronay Ak, and Even Oldridge. Transformers4rec: Bridging the gap between nlp and sequential/session-based recommendation. In *Fifteenth ACM Conference on Recommender Systems*, pages 143–153, 2021.

- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [8] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). *arXiv preprint arXiv:2203.13366*, 2022.
- [9] Carlos A Gomez-Urbe and Neil Hunt. The netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems (TMIS)*, 6(4):1–19, 2015.
- [10] Ruining He and Julian McAuley. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *proceedings of the 25th international conference on world wide web*, pages 507–517, 2016.
- [11] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939*, 2015.
- [12] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. Learning vector-quantized item representation for transferable sequential recommenders. *arXiv preprint arXiv:2210.12316*, 2022.
- [13] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*, pages 604–613, 1998.
- [14] Piotr Indyk, Rajeev Motwani, Prabhakar Raghavan, and Santosh Vempala. Locality-preserving hashing in multidimensional spaces. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pages 618–625, 1997.
- [15] Herve Jegou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence*, 33(1):117–128, 2010.
- [16] Wang-Cheng Kang, Derek Zhiyuan Cheng, Tiansheng Yao, Xinyang Yi, Ting Chen, Lichan Hong, and Ed H Chi. Learning to embed categorical features without embedding tables for recommendation. *arXiv preprint arXiv:2010.10784*, 2020.
- [17] Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pages 197–206. IEEE, 2018.

- [18] Dongmoon Kim, Kun-su Kim, Kyo-Hyun Park, Jee-Hyong Lee, and Keon Myung Lee. A music recommendation system with a dynamic k-means clustering algorithm. In *Sixth international conference on machine learning and applications (ICMLA 2007)*, pages 399–403. IEEE, 2007.
- [19] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [20] Tim Kraska, Alex Beutel, Ed H Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *Proceedings of the 2018 international conference on management of data*, pages 489–504, 2018.
- [21] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11523–11532, 2022.
- [22] Hyunji Lee, Jaeyoung Kim, Hoyeon Chang, Hanseok Oh, Sohee Yang, Vlad Karpukhin, Yi Lu, and Minjoon Seo. Contextualized generative retrieval. *arXiv preprint arXiv:2210.02068*, 2022.
- [23] Hyunji Lee, Sohee Yang, Hanseok Oh, and Minjoon Seo. Generative retrieval for long sequences. *arXiv preprint arXiv:2204.13596*, 2022.
- [24] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. Neural attentive session-based recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pages 1419–1428, 2017.
- [25] Chen Ma, Peng Kang, and Xue Liu. Hierarchical gating networks for sequential recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 825–833, 2019.
- [26] Masoud Mansoury, Himan Abdollahpouri, Mykola Pechenizkiy, Bamshad Mobasher, and Robin Burke. Feedback loop and bias amplification in recommender systems, 2020.
- [27] Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith Hall, Daniel Cer, and Yinfei Yang. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 1864–1874, Dublin, Ireland, May 2022. Association for Computational Linguistics.
- [28] Adam Roberts, Hyung Won Chung, Anselm Levskaya, Gaurav Mishra, James Bradbury, Daniel Andor, Sharan Narang, Brian Lester, Colin Gaffney, Afroz Mohiuddin, Curtis Hawthorne, Aitor Lewkowycz, Alex Salcianu, Marc van Zee,

- Jacob Austin, Sebastian Goodman, Livio Baldini Soares, Haitang Hu, Sasha Tsvyashchenko, Aakanksha Chowdhery, Jasmijn Bastings, Jannis Bulian, Xavier Garcia, Jianmo Ni, Andrew Chen, Kathleen Kenealy, Jonathan H. Clark, Stephan Lee, Dan Garrette, James Lee-Thorp, Colin Raffel, Noam Shazeer, Marvin Ritter, Maarten Bosma, Alexandre Passos, Jeremy Maitin-Shepard, Noah Fiedel, Mark Omernick, Brennan Saeta, Ryan Sepassi, Alexander Spiridonov, Joshua Newlan, and Andrea Gesmundo. Scaling up models and data with `t5x` and `seqio`. *arXiv preprint arXiv:2203.17189*, 2022.
- [29] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [30] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics.
- [31] Anima Singh, Trung Vu, Raghunandan Keshavan, Nikhil Mehta, Xinyang Yi, Lichan Hong, Lukasz Heldt, Li Wei, Ed Chi, and Maheswaran Sathiamoorthy. Better generalization with semantic ids: A case study in ranking for recommendations. *arXiv preprint arXiv:2306.08121*, 2023.
- [32] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 1441–1450, 2019.
- [33] Jiayi Tang and Ke Wang. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*, pages 565–573, 2018.
- [34] Yi Tay, Vinh Q Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, et al. Transformer memory as a differentiable search index. *arXiv preprint arXiv:2202.06991*, 2022.
- [35] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [37] Yujing Wang, Yingyan Hou, Haonan Wang, Ziming Miao, Shibin Wu, Hao Sun, Qi Chen, Yuqing Xia, Chengmin Chi, Guoshuai Zhao, et al. A neural corpus indexer for document retrieval. *arXiv preprint arXiv:2206.02743*, 2022.

- [38] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 1113–1120, 2009.
- [39] Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi. Sampling-bias-corrected neural modeling for large corpus item recommendations. In *Proceedings of the 13th ACM Conference on Recommender Systems*, pages 269–277, 2019.
- [40] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. Soundstream: An end-to-end neural audio codec. *CoRR*, abs/2107.03312, 2021.
- [41] Shuai Zhang, Yi Tay, Lina Yao, and Aixin Sun. Next item recommendation with self-attention. *arXiv preprint arXiv:1808.06414*, 2018.
- [42] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, and Xiaofang Zhou. Feature-level deeper self-attention network for sequential recommendation. In *IJCAI*, pages 4320–4326, 2019.
- [43] Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, and Ed Chi. Recommending what video to watch next: a multitask ranking system. In *Proceedings of the 13th ACM Conference on Recommender Systems*, pages 43–51, 2019.
- [44] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 1893–1902, 2020.

A 相关工作 (续)

生成式检索 传统的文档检索通常训练一个双塔模型，将查询和文档映射到同一高维向量空间，随后针对查询在所有文档上执行 ANN 或 MIPS，以返回最接近的文档。这项技术存在一些缺点，例如需要大型嵌入表 [22, 23]。生成式检索是近期提出的一项技术，旨在通过逐词元生成文档的标题、名称或文档 ID 字符串，解决传统方法的一些问题。Cao 等人 [5] 提出了用于实体检索的 GENRE，它采用基于 Transformer 的架构，逐词元返回给定查询所指实体的名称。Tay 等人 [34] 提出了用于文档检索的 DSI，这是首个为每篇文档分配结构化语义 DocID 的系统。给定一个查询后，模型以自回归方式逐词元返回相应文档的 DocID。DSI 的工作标志着信息检索向生成式检索方法的范式转变，也是端到端 Transformer 在检索应用中的首次成功实践。随后，Lee 等人 [23] 表明，即使在多跳场景中，生成式文档检索也很有用；在该场景下，复杂查询无法由单篇文档直接回答，因此其模型以思维链方式生成中间查询，最终生成复杂查询的输出。Wang 等人 [37] 针对 Tay 等人 [34] 基于层次化 k -means 聚类的语义 DocID，提出了一种专门考虑语义 DocID 前缀的新型解码器架构作为补充。在 CGR [22] 中，作者提出了一种同时利用双编码器技术和生成式检索技术的方法：让其基于编码器-解码器的模型中的解码器学习独立的上下文嵌入，从而在嵌入内部存储文档信息。据我们所知，我们首次将使用自编码器 (RQ-VAE [40, 21]) 创建的生成式语义 ID 用于检索模型。

向量量化。 我们将向量量化定义为把高维向量转换成低维码字元组的过程。最直接的技术之一是使用层次聚类，例如 [34] 中所采用的方法：某次迭代中生成的聚类会在下一次迭代中进一步划分为子聚类。另一种常用方法是向量量化变分自编码器 (VQ-VAE)，[35] 首次将其作为一种把自然图像编码为编码序列的方法提出。该技术首先将输入向量 (或图像) 送入编码器以降低维度。随后将较低维的向量分块，并分别量化每个分块，从而得到一个编码序列，即每个分块对应一个编码。解码器再使用这些编码重建原始向量 (或图像)。

RQ-VAE [40, 21] 对 VQ-VAE 编码器的输出应用残差量化，以获得更低的重建误差。我们在第 3.1 小节中更详细地讨论该技术。局部敏感哈希 (LSH) [14, 13] 是一种广泛用于聚类和近似最近邻搜索的技术。本文用于聚类的具体版本是 SimHash [2]，它使用随机超平面生成二进制向量，作为物品的哈希值。由于该方法计算复杂度低且可扩展 [13]，我们将其用作向量量化的基线技术。

B 基线方法

下面简要介绍用于与 TIGER 比较的各个基线

- GRU4Rec [11] 是首个基于 RNN 的方法，它使用定制的 GRU 执行序列推荐任务。
- Caser [33] 采用 CNN 架构，通过应用水平和垂直卷积操作捕捉高阶马尔可夫链，以进行序列推荐。
- HGN [25]：分层门控网络通过一种新的门控架构捕捉用户的长期和短期兴趣。

- SASRec [17]: 自注意力序列推荐使用带因果掩码的 Transformer 对用户的序列交互进行建模。
- BERT4Rec [32]: BERT4Rec 使用双向自注意力 Transformer 执行推荐任务，从而解决单向架构的局限性。
- FDSA [42]: 特征级深层自注意力网络除物品嵌入外，还将物品特征作为 Transformer 输入序列的一部分。
- S³-Rec [44]: 序列推荐的自监督学习提出在自监督任务上预训练双向 Transformer，以改善序列推荐。
- P5 [8]: P5 是一种近期方法，它使用预训练大语言模型（LLM）在单一模型中统一不同的推荐任务。

C 数据集统计

表 6: 三个真实世界基准数据集的统计信息。

数据集	# 用户	# 物品	序列长度	
			均值	中位数
美容	22,363	12,101	8.87	6
运动与户外	35,598	18,357	8.32	6
玩具与游戏	19,412	11,924	8.63	6

我们使用 Amazon Product Reviews 数据集 [10] 中的三个公开基准，其中包含 1996 年 5 月至 2014 年 7 月期间的用户评论和物品元数据。针对序列推荐任务，我们选用该数据集的三个类别：“美容”、“运动与户外”和“玩具与游戏”。表 6 汇总了这些数据集的统计信息。我们使用用户的评论历史创建按时间戳排序的物品序列，并过滤掉评论数少于 5 条的用户。遵循标准评估协议 [17, 8]，我们采用留一法进行评估。对于每个物品序列，最后一个物品用于测试，倒数第二个物品用于验证，其余物品用于训练。在训练期间，我们将用户历史中的物品数量限制为 20。

D 对 P5 数据预处理的修改

表 7: 采用标准预处理的 P5[8] 结果。

方法	运动与户外				美容				玩具与游戏			
	Recall@5	NDCG@5	Recall@10	NDCG@10	Recall@5	NDCG@5	Recall@10	NDCG@10	Recall@5	NDCG@5	Recall@10	NDCG@10
P5	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
P5-ours	0.0107	0.0076	0.01458	0.0088	0.035	0.025	0.048	0.0298	0.018	0.013	0.0235	0.015

表 8: 向推荐系统提供用户信息的影响

Recall@5	NDCG@5	Recall@10	NDCG@10	
无用户信息	0.04458	0.0302	0.06479	0.0367
含用户 ID (论文报告结果)	0.0454	0.0321	0.0648	0.0384

P5 源代码⁵对 Amazon 数据集进行预处理, 首先为每位用户创建会话, 其中包含该用户评论过的物品按时间顺序排列的列表。创建这些会话后, 数据集中的原始物品 ID 被重新映射为整数 $1, 2, 3, \dots$ ⁶。因此, 第一个会话中的第一个物品获得 ID ‘1’, 第二个物品若此前未出现则获得 ID ‘2’, 依此类推。值得注意的是, 这一预处理方案在创建训练集和测试集划分之前应用。这会产生许多形如 $a, a+1, a+2, \dots$ 的序列。由于 P5 使用 Sentence Piece 分词器 [30] (见 [8] 的第 4.1 节), 用户会话中的测试物品和训练物品可能共享子词, 从而在推理阶段导致信息泄漏。

为解决这一泄漏问题, 我们不再为物品分配连续递增的整数 ID, 而是分配随机整数 ID, 之后再创建训练和评估划分。P5 的其余代码与论文提供的源代码保持一致。该数据集上的结果在表 7 的 ‘P5’ 行中报告。我们还从头自行实现了一个 P5 版本, 并且仅在序列推荐任务上训练模型。我们实现的结果以 ‘P5-ours’ 表示。我们还在自己的 P5 实现中验证了, 物品 ID 使用连续整数序列能够获得与 P5 报告结果相当或更好的指标。

E 讨论

表 9: 不同数据集上各指标的均值和标准误 (使用 3 个不同随机种子运行所得)

数据集	Recall@5	NDCG@5	Recall@10	NDCG@10
Beauty	0.0441 ± 0.00069	0.0309 ± 0.00062	0.0642 ± 0.00092	0.0374 ± 0.00061
Sports and Outdoors	0.0278 ± 0.00069	0.0189 ± 0.00043	0.0419 ± 0.0010	0.0234 ± 0.00048
Toys and Games	0.0518 ± 0.00064	0.0375 ± 0.00039	0.0698 ± 0.0013	0.0433 ± 0.00047

语义 ID 长度和码本大小的影响。我们尝试改变语义 ID 的长度和码本大小, 例如令 ID 由 6 个码字组成, 每个码字均来自大小为 64 的码本。我们发现, TIGER 的推荐指标对这些变化具有稳健性。但需要注意的是, ID 越长 (即每个物品 ID 包含的码字越多), 输入序列也越长, 这会增加基于 Transformer 的序列到序列模型的计算成本。

可扩展性。为测试语义 ID 的可扩展性, 我们进行了以下实验: 将三个数据集合并, 并为三个数据集中的全部物品生成语义 ID。随后, 我们使用这些语义 ID 在 Beauty 数据集上执行推荐任务。我们将该实验的结果与仅使用 Beauty 数据集生成语义 ID 的原始实验进行比较。结果见表 10。可以看出, 此处的性能仅有小幅下降。

⁵<https://github.com/jeykigung/P5>

⁶https://github.com/jeykigung/P5/blob/0aaa3fd8366bb6e708c8b70708291f2b0ae90c82/preprocess/data_preprocess_amazon.ipynb

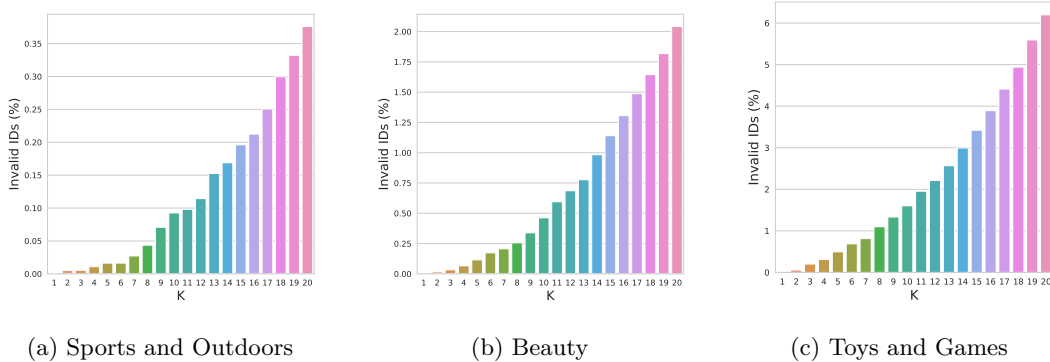


图 6: 使用束搜索生成语义 ID 时, 不同 K 取值对应的无效 ID 百分比。如图所示, 在检索排名前 20 的物品时, 有 $\sim 0.3\% - 6\%$ 的 ID 无效。

表 10: 通过对比在合并数据集上生成语义 ID 与仅在 Beauty 数据集上生成语义 ID 来测试可扩展性。

	Recall@5	NDCG@5	Recall@10	NDCG@10
语义 ID [合并数据集]	0.04355	0.3047	0.06314	0.03676
语义 ID [Amazon Beauty]	0.0454	0.0321	0.0648	0.0384

推理成本。尽管我们的模型在序列推荐任务上取得了显著成功, 但我们注意到, 由于使用束搜索进行自回归解码, 在推理过程中, 我们的模型可能比基于 ANN 的模型需要更多计算资源。我们强调, 优化 TIGER 的计算效率并非本工作的主要目标。相反, 我们的工作开辟了一个新的研究领域: 基于生成式检索的推荐系统。作为未来工作的一部分, 我们将考虑缩小模型的方法, 或探索其他提升推理效率的途径。

查找表的内存成本。我们为 TIGER 维护两个哈希查找表: 物品 ID 到语义 ID 的表, 以及语义 ID 到物品 ID 的表。请注意, 这两个表都只生成一次, 之后便会冻结: 它们在基于 RQ-VAE 的语义 ID 生成模型训练完成后生成, 随后在序列到序列 Transformer 模型的训练过程中保持冻结。每个语义 ID 由 4 个整数组成的元组构成, 每个整数用 8 位存储, 因此每个物品共需 32 位。每个物品由一个物品 ID 表示, 该 ID 存储为 32 位整数。因此, 每个查找表的大小为 $64N$ 位量级, 其中 N 是数据集中的物品数量。

嵌入表的内存成本。与传统推荐系统相比, TIGER 使用的嵌入表小得多。这是因为传统推荐系统为每个物品存储一个嵌入, 而 TIGER 仅为每个语义码字存储一个嵌入。在实验中, 我们使用 4 个基数均为 256 的码字表示语义 ID, 因此共有 1024 (256×4) 个嵌入。对于传统推荐系统, 嵌入数量为 N , 其中 N 是数据集中的物品数量。在实验中, 根据数据集的不同, N 的取值范围为 10K 到 20K。因此, TIGER 嵌入表的内存成本为 $1024d$, 其中 d 是嵌入维度; 而传统推荐系统中嵌入查找表的内存成本为 Nd 。